

Backups and Restore: A Boring, Tested, Offsite Strategy for a Personal VPS

Backups that you've actually tested, stored somewhere outside your VPS, automated end-to-end, and quick enough to restore that you'd reach for them at 11pm when something has gone wrong. Built on top of [vps-from-zero](#)'s shared Postgres setup.

NOTE. LAST VALIDATED: 2026-05. Postgres 16's `pg_dumpall`/`pg_dump` toolchain, rclone current (1.65+), Backblaze B2 current pricing (\$0.005/GB-month storage, \$0.01/GB egress), age 1.2+ for encryption. Bump and re-validate annually per [CLAUDE.md](#) standard #5.

What you'll have at the end

- A nightly automated **Postgres dump** stored locally on the VPS with reasonable retention (14 days).
- The same dumps **encrypted and synced offsite** to Backblaze B2 (or any S3-compatible storage) — costs about \$0.05/month for a few GB.
- An offsite copy of your **Caddy configuration and any per-app `.env` files** so you can rebuild from scratch.
- A documented **restore-from-disaster procedure** that gets you from "VPS is gone" to "everything works" in under an hour.
- A **monthly restore drill** that catches "the backup didn't actually work" before you need it.
- The discipline of not backing up things you shouldn't (logs, Docker images, `.git` contents that already live on GitHub).

Prerequisites

- A working VPS from [vps-from-zero](#). Specifically you need the `infra/postgres` container running.
 - 5 minutes of attention to your Backblaze account (free signup, no card required for testing the API).
 - A spare ~30 minutes for the first setup. ~10 minutes/year for the monthly drill thereafter.
-

Table of contents

1. [The principles](#)
2. [What to back up \(and what not to\)](#)
3. [Local backup: the script and the cron](#)
4. [Offsite: Backblaze B2 + rclone](#)
5. [Encryption at rest with `age`](#)
6. [Backing up Caddy config and `.env` files](#)

- 7. [Restoring a database](#)
 - 8. [Disaster recovery: rebuild from zero](#)
 - 9. [The monthly restore drill](#)
 - 10. [Retention policy](#)
 - 11. [Troubleshooting](#)
 - 12. [Alternatives considered](#)
 - 13. [Quick reference](#)
-

1. The principles

1. **A backup that you've never restored from is a wish, not a backup.** The single most common failure mode is "the backup script ran every night for years but produced corrupt dumps that no one verified." Test restores at least monthly.
 2. **Offsite or it doesn't count.** A backup that lives on the same machine as the data — even on a different disk — is gone the moment the machine is destroyed. Local backups are a first tier (fast restore from "oops I dropped a table"), but offsite is what saves you when the box catches fire.
 3. **Encrypt at rest.** Your backups contain everything an attacker who breaches your storage provider could want. Encrypt with a key whose private half is not stored anywhere your backups go.
 4. **Restore is the operation that matters.** The procedure should be 5-10 commands, runnable from memory or from a saved doc you can find without internet. Anything more elaborate is a procedure you won't actually do at 2am.
 5. **Don't back up what GitHub / the registry already has.** Source code is in git, container images are in GHCR, both are independently durable. Backups are for **derived state** — databases, user uploads, certificates, configs.
 6. **Automate everything, then forget it.** Manual backups don't happen consistently. Automated backups happen at 3am whether you remember or not.
-

2. What to back up (and what not to)

Data	Back up?	Where it lives	Notes
Postgres databases	☐ Yes	<code>infra_postgres_data</code> Docker volume	The crown jewel. Dump nightly.
App user uploads	☐ Yes	Per-app Docker volumes	If your app stores files (Discord bot attachments, uploaded images, etc.).
App <code>.env</code> files	☐ Yes (encrypted)	<code>/home/deploy/<app>/.env</code>	Has secrets. Encrypt before storing.
Caddy data (certs)	☐ Yes	<code>caddy_data</code> Docker volume	So you don't hit LE rate limits during a restore.
Caddyfile	☐ Yes	<code>/home/deploy/infra/Caddyfile</code>	Trivially small; back up.
Cron entries	☐ Yes	<code>/etc/cron.d/</code>	Mostly the backup cron itself.
Application source code	× No	GitHub	Already replicated; restoring is <code>git clone</code> .
Container images	× No	GHCR	Same. Restore is <code>docker pull</code> .
System packages	× No	apt	Reinstallable via the <code>vps-from-zero</code> runbook.
Docker logs	× No	Container filesystems	Ephemeral by design; rotated by Docker.
<code>/tmp</code> , <code>/var/cache</code> , <code>~/.cache</code>	× No	Various	Ephemeral.
SSH host keys	△ Maybe	<code>/etc/ssh/ssh_host_*</code>	If you back them up, restoring a "new" VPS keeps the same host fingerprint — no "host key changed" warnings. Some people consider this a feature; others a security regression. Pick.

The pattern: **back up derived state** (databases, configs, user uploads). Don't back up **reproducible artifacts** (code, images, packages).

3. Local backup: the script and the cron

The basic shape: a shell script that dumps Postgres, gzip-compresses, places the file in `~/backups/` on the VPS, and prunes old files. Run via cron.

The script

```
/home/deploy/scripts/backup-db.sh:
```

```
#!/usr/bin/env bash
set -euo pipefail

# Where to store backups on the VPS.
BACKUP_DIR=/home/deploy/backups
mkdir -p "$BACKUP_DIR"

# Time-stamped filename. UTC for sortability across timezones.
TIMESTAMP=$(date -u +%Y%m%dT%H%M%SZ)
DEST="$BACKUP_DIR/pgdump-$TIMESTAMP.sql.gz"

# Dump all databases + global state (users, roles).
docker exec infra-postgres-1 pg_dumpall -U postgres | gzip -9 > "$DEST"

# Verify the dump is non-empty (a 0-byte dump indicates failure that
# pg_dumpall didn't propagate as an exit code, e.g., container died
# mid-stream).
if [ ! -s "$DEST" ]; then
    echo "ERROR: backup file is empty: $DEST" >&2
    rm -f "$DEST"
    exit 1
fi

# Local retention: keep last 14 days.
find "$BACKUP_DIR" -name 'pgdump-*.sql.gz' -mtime +14 -delete

# Log success with size for sanity checking
size=$(du -h "$DEST" | cut -f1)
echo "$(date -u +%FT%TZ) backup OK: $DEST ($size)"
```

Make it executable:

```
chmod +x /home/deploy/scripts/backup-db.sh
```

Why `pg_dumpall` and not `pg_dump`

`pg_dump` dumps **one database** as schema + data SQL. `pg_dumpall` dumps **all databases plus global state** (roles, users, passwords, tablespaces). For a personal VPS hosting multiple apps that share one Postgres instance, you want `pg_dumpall` — recreating roles from scratch after a restore is tedious if you forget about it.

The downside: `pg_dumpall` produces a single big file that has to be restored as a unit. For massive databases you'd use `pg_dump` per database (in parallel). At personal scale, this never matters.

Why `gzip -9`

Postgres dumps are extraordinarily compressible — lots of repeated SQL keywords and structured data. `gzip -9` (max compression) is CPU-cheap relative to the size savings on dumps. 100MB → 8MB is typical.

Why explicitly check file size

Sometimes `docker exec` fails silently (the container died mid-stream), and you get a partial or empty file with no error. The `[ ! -s "$DEST" ]` check catches this. A backup-script failure should fail loudly, not silently leave a corrupt file.

The cron entry

As root, create `/etc/cron.d/pg-backup`:

```
# Run nightly at 03:17 UTC, as the deploy user.
17 3 * * * deploy /home/deploy/scripts/backup-db.sh >> /var/log/pg-backup.log 2>&1
```

A few details:

- **17 minutes past the hour**: arbitrary offset to avoid the synchronized-overload-at-:00 effect.
- **3am UTC**: low-traffic hour for most apps. Adjust to whenever your traffic is lowest.
- **deploy user**: the user with docker access. Runs the script with the right permissions.
- **Append-redirect to a log**: so cron failures don't email-spam root. The log can be tail'd when needed.

Verify the script works

Don't wait for cron. Run it manually first:

```
sudo -iu deploy /home/deploy/scripts/backup-db.sh
ls -lh ~/backups/
# Should see a recent pgdump-*.sql.gz
```

If anything fails, you'll see it immediately.

4. Offsite: Backblaze B2 + rclone

The local script gives you "oops I dropped a table" recovery (fast, on-VPS). Now you need "the VPS is gone" recovery — backups in a different location, on someone else's infrastructure.

Backblaze B2 is the cheapest credible offsite storage. At \$0.005/GB-month, storing 10GB of backups costs \$0.05/month — five cents. Egress is \$0.01/GB; restoring 10GB costs ten cents. The math is irrelevant compared to losing data.

One-time setup

1. **Sign up** at backblaze.com (free, no credit card needed to start).
2. **Create a bucket**. Backblaze console → B2 Cloud Storage → Buckets → Create. Name it `pgdumps-yourdomain` or similar. Files are private. Lifecycle: keep all versions.
3. **Generate an application key** with read+write to that bucket. Save the keyID and applicationKey somewhere (1Password).
4. **Install rclone on the VPS** (it's in apt; alternatively, `curl https://rclone.org/install.sh | sudo bash` for latest):

```
bash sudo apt install rclone
```

1. **Configure rclone** (as the deploy user):

```
bash sudo -iu deploy rclone config
```

Walk through: - `n` for new remote - Name: `b2` - Storage: `b2` (Backblaze B2) - Account: your keyID - Key: your applicationKey - Skip endpoint - `n` for advanced config - `y` to confirm

This writes credentials to `/home/deploy/.config/rclone/rclone.conf`. Set tight permissions:

```
bash chmod 600 /home/deploy/.config/rclone/rclone.conf
```

1. **Test:**

```
bash rclone lsf b2:pgdumps-yourdomain # (empty if the bucket is empty – should not error)
```

Add offsite to the backup script

Update `/home/deploy/scripts/backup-db.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

BACKUP_DIR=/home/deploy/backups
mkdir -p "$BACKUP_DIR"

TIMESTAMP=$(date -u +%Y%m%dT%H%M%S)
DEST="$BACKUP_DIR/pgdump-`${TIMESTAMP}.sql.gz"

docker exec infra-postgres-1 pg_dumpall -U postgres | gzip -9 > "$DEST"

if [ ! -s "$DEST" ]; then
    echo "ERROR: backup file is empty: $DEST" >&2
    rm -f "$DEST"
    exit 1
fi

# Local retention
find "$BACKUP_DIR" -name 'pgdump-*.sql.gz' -mtime +14 -delete

# Offsite upload (only the new file; rclone is idempotent so resyncs are safe)
rclone copy "$DEST" b2:pgdumps-yourdomain/postgres/ --quiet

# Offsite retention: keep last 90 days (handled by --max-age)
rclone delete b2:pgdumps-yourdomain/postgres/ --min-age 90d --quiet

size=$(du -h "$DEST" | cut -f1)
echo "${date -u +%FT%TZ} backup + sync OK: $DEST ($size)"
```

Two new behaviors:

- `rclone copy` uploads the new dump to B2.
- `rclone delete --min-age 90d` removes dumps older than 90 days from B2.

Local retention is 14 days; offsite is 90 days. The asymmetry is intentional: local backups are for fast same-day recovery; offsite is for "did something silently corrupt 60 days ago?" recovery.

Bandwidth and frequency

For a personal VPS with a small Postgres database:

- Daily dumps are ~10MB compressed (typical).
- Daily B2 cost: ~\$0.0015 storage + ~\$0.0001 transfer = ~\$0.05/month total for the year's worth.

If your DB grows, scale up the cron interval (still nightly is fine to ~10GB) or use incremental backups (next-tier complexity, not needed for personal scale).

5. Encryption at rest with `age`

The dumps in B2 contain everything — all user data, all app `.env` secrets that have made it into the DB, all session tokens. **Encrypt the dumps before they leave the VPS**, with a key whose private half is not stored in B2.

`age` is the modern, simple file encryption tool. Smaller surface area than GPG, no key servers, just a public-key crypto primitive that works.

Set up `age`

```
sudo apt install age # or download from the GitHub releases for latest
age-keygen -o ~/.config/age/keys.txt
chmod 600 ~/.config/age/keys.txt
```

The file contains a private key and a corresponding public key (printed to stderr during keygen — also visible inside the file). Looks like:

```
# created: 2026-05-27T15:30:00Z
# public key: age1abc123...xyz789
AGE-SECRET-KEY-1MNOPQR...
```

IMPORTANT. STORE THE PRIVATE KEY (`~/.CONFIG/AGE/KEYS.TXT`) SOMEWHERE OUTSIDE THE VPS. Copy it to a password manager (1Password, Bitwarden) or a separate device. If you only have it on the VPS, and the VPS dies, you can't decrypt the offsite backups. Catastrophic.

Update the backup script

```
#!/usr/bin/env bash
set -euo pipefail

BACKUP_DIR=/home/deploy/backups
AGE_RECIPIENT="age1abc123...xyz789" # your public key (safe to embed in script)

mkdir -p "$BACKUP_DIR"

TIMESTAMP=$(date -u +%Y%m%dT%H%M%SZ)
DEST="$BACKUP_DIR/pgdump-$TIMESTAMP.sql.gz.age"

# Dump → gzip → age encrypt → write
docker exec infra-postgres-1 pg_dumpall -U postgres \
    | gzip -9 \
    | age -r "$AGE_RECIPIENT" -o "$DEST"

if [ ! -s "$DEST" ]; then
    echo "ERROR: backup file is empty: $DEST" >&2
    rm -f "$DEST"
    exit 1
fi

find "$BACKUP_DIR" -name 'pgdump-*.sql.gz.age' -mtime +14 -delete

rclone copy "$DEST" b2:pgdumps-yourdomain/postgres/ --quiet
rclone delete b2:pgdumps-yourdomain/postgres/ --min-age 90d --quiet

size=$(du -h "$DEST" | cut -f1)
echo "$(date -u +%FT%TZ) encrypted backup OK: $DEST ($size)"
```

The change is the `| age -r "$AGE_RECIPIENT" -o "$DEST"` step. age encrypts the stream using your public key; the result is decryptable only with the matching private key.

`-r` (recipient) takes a public key. The public key is safe to embed in the script — that's the whole point of public-key crypto.

Decrypting

When restoring (more in §7):

```
age -d -i ~/.config/age/keys.txt pgdump-XXX.sql.gz.age | gunzip > restored.sql
```

`-d` for decrypt; `-i` for identity file (the private key).

Multi-recipient encryption

You can encrypt to multiple recipients (more public keys). Useful if your spouse or co-founder should also be able to decrypt:

```
age -r age1abc... -r age1def... -o output.age
```

A common pattern: a key on your laptop AND a key in 1Password's secure note. If your laptop dies, you can recover via the 1Password copy.

6. Backing up Caddy config and `.env` files

Postgres is the crown jewel, but a few other small things matter for full disaster recovery.

What's needed for a full rebuild

- `~/infra/docker-compose.yml` and `~/infra/.env` (infra stack config)
- `~/infra/Caddyfile` (reverse proxy routes)
- `~/infra/caddy_data/` (cached Let's Encrypt certs — avoids re-issuing all certs on restore, which would hit rate limits if you have many)
- `~/<app>/docker-compose.yml` and `~/<app>/.env` for each app

A simple tarball approach:

```
#!/usr/bin/env bash
# /home/deploy/scripts/backup-configs.sh
set -euo pipefail

BACKUP_DIR=/home/deploy/backups
AGE_RECIPIENT="age1abc123..."
TIMESTAMP=$(date -u +%Y%m%dT%H%M%SZ)
DEST="$BACKUP_DIR/configs-$TIMESTAMP.tar.gz.age"

# Build the tarball
tar -czf - \
  -C /home/deploy \
  --exclude='*/postgres_data' \
  --exclude='*/.git' \
  infra/ \
  $(ls -d /home/deploy/*/ 2>/dev/null | grep -v 'backups|scripts|.config' | xargs -n1 basename) \
  | age -r "$AGE_RECIPIENT" -o "$DEST"

rclone copy "$DEST" b2:pgdumps-yourdomain/configs/ --quiet
rclone delete b2:pgdumps-yourdomain/configs/ --min-age 90d --quiet

# Local retention: keep last 7 (these change slowly)
ls -t "$BACKUP_DIR"/configs-*.tar.gz.age 2>/dev/null | tail -n +8 | xargs -r rm
```

Add to `/etc/cron.d/pg-backup`:

```
17 3 * * * deploy /home/deploy/scripts/backup-db.sh >> /var/log/pg-backup.log 2>&1
27 3 * * 0 deploy /home/deploy/scripts/backup-configs.sh >> /var/log/pg-backup.log 2>&1
```

The configs backup runs weekly (Sundays at 03:27) — configs don't change daily.

NOTE. THE `CADDY_DATA` VOLUME IS BIND-MOUNTED INTO THE CONTAINER, BUT IF YOU USED A NAMED VOLUME INSTEAD, YOU'LL NEED TO DUMP IT VIA `DOCKER RUN --RM -V CADDY_DATA:/DATA ALPINE TAR -CZF - /DATA | AGE -R ...`. Bind mounts (showing up in `~/infra/caddy_data/`) are simpler for backups.

7. Restoring a database

The common case: "I just deleted a table I shouldn't have" or "I need to roll back to before this migration."

Restore to the same VPS

```
# 1. Identify the backup you want
ls -la ~/backups/
# pgdump-20260520T031700Z.sql.gz.age – restore to this point

# 2. Decrypt to a working file
age -d -i ~/.config/age/keys.txt \
  ~/backups/pgdump-20260520T031700Z.sql.gz.age \
  | gunzip > /tmp/restore.sql

# 3. (Optional but recommended) Stop the apps that connect to the DB
cd ~/apm-bot && docker compose stop
cd ~/dashboard && docker compose stop

# 4. Restore. pg_dumpall produces SQL that recreates roles and dbs,
# so we connect as the postgres superuser to the postgres database.
docker exec -i infra-postgres-1 psql -U postgres < /tmp/restore.sql

# 5. Remove the working file (contains plaintext data)
shred -u /tmp/restore.sql

# 6. Start the apps
cd ~/apm-bot && docker compose start
cd ~/dashboard && docker compose start
```

Restore to a fresh Postgres instance

Same as above, but starting from an empty `postgres` container:

```
cd ~/infra
docker compose up -d postgres
# wait for healthcheck
docker compose ps postgres # should show "healthy"

# Then steps 2 + 4 from above
```

`pg_dumpall` output includes `CREATE ROLE` and `CREATE DATABASE` statements, so a fresh instance ends up with all the same users, passwords, and databases as the source. No need to manually re-create the per-app users.

Restore a single database (not all)

If you have a `pg_dumpall` and only want one database:

```
# Decrypt, extract just one db's section, restore
age -d -i ~/.config/age/keys.txt pgdump-XXX.sql.gz.age | gunzip > /tmp/all.sql

# pg_dumpall produces a file with section markers; use grep/awk to extract
# But simpler: use pg_dump per-db at backup time if this is a common case

# Or use psql's interactive mode to surgically rebuild just the needed pieces
```

For "I'll occasionally need per-db restores," consider switching to `pg_dump --create -d <dbname>` per database in your backup script. Tradeoff: more files to manage. For personal use, `pg_dumpall` is fine — restoring everything is rarely a problem.

Point-in-time recovery

For "I want to restore to 14:32 yesterday, not just last night's snapshot": Postgres supports continuous archiving (WAL streaming). Setup is more complex; for personal use, nightly snapshots are usually enough. Reconsider PITR if your data is high-velocity and 24 hours of loss would be painful.

8. Disaster recovery: rebuild from zero

The VPS is gone — disk failure, provider-side outage, accidental `rm -rf /`. You need to restore service onto a new VPS.

The runbook

1. **Provision a fresh VPS** per `vps-from-zero`. 20-30 minutes.
2. **Install rclone** and configure with the same B2 keys: `bash sudo apt install rclone sudo -iu deploy rclone config # re-add the b2 remote`
3. **Restore the age private key.** Pull it from 1Password (or wherever you stored it outside the VPS) and write to `/home/deploy/.config/age/keys.txt`: `bash mkdir -p /home/deploy/.config/age # paste private key content chmod 600 /home/deploy/.config/age/keys.txt`
4. **Pull the latest configs backup:** `bash sudo -iu deploy rclone copy b2:pgdumps-yourdomain/configs/ ~/restore/ --max-age 7d ls ~/restore/`
5. **Decrypt and extract:** `bash cd ~ age -d -i ~/.config/age/keys.txt ~/restore/configs-XXX.tar.gz.age | tar -xzf -`

This restores `~/infra/`, `~/<app1>/`, `~/<app2>/`, etc.

1. **Create the shared Docker network:** `bash docker network create shared`
2. **Bring up the infra stack:**
`bash cd ~/infra docker compose up -d postgres docker compose ps # wait for healthy`
3. **Pull the latest Postgres dump and restore:** `bash rclone copy b2:pgdumps-yourdomain/postgres/ ~/restore/ --max-age 2d age -d -i ~/.config/age/keys.txt ~/restore/pgdump-XXX.sql.gz.age \ | gunzip \ | docker exec -i infra-postgres-1 psql -U postgres`
4. **Bring up Caddy and the apps:** `bash cd ~/infra docker compose up -d caddy`

for app in apm-bot dashboard; do cd ~/\$app docker compose pull docker compose up -d done ``

1. **Update DNS** to point at the new VPS's IP. Browsers and bots will start hitting the new box within seconds (Cloudflare's 5-minute TTL) to a few hours (depending on your DNS provider).

Total time: 30-60 minutes for someone who's done it once before. The first run is closer to 90 minutes including DNS propagation.

What goes wrong in disaster recovery

- **You don't have the age private key.** All offsite backups are useless. This is the failure mode this guide is trying to prevent. **Store the private key outside the VPS.**
 - **You don't remember which Backblaze account / bucket.** Save the keyID + bucket name in 1Password too.
 - **Cloudflare DNS hasn't propagated.** TTL is usually 5 minutes; rarely longer. Wait it out.
 - **Let's Encrypt rate-limits hit.** If you didn't back up the `caddy_data` volume, Caddy will try to re-issue all certs at once. The hourly rate limit (5 failed validations per hour) gets hit. Either: back up `caddy_data` (preferred), or stagger Caddy's start by adding sites to Caddyfile one at a time.
-

9. The monthly restore drill

A backup you've never restored is theoretical. **Once a month, restore your most recent dump to a throwaway environment and verify it works.**

The drill (10 minutes once a month)

On your laptop, in a clean directory:

```

mkdir /tmp/drill && cd /tmp/drill

# 1. Pull the latest dump from B2
rclone copy b2:pgdumps-yourdomain/postgres/ . --max-age 2d

# 2. Decrypt and unzip
age -d -i ~/.config/age/keys.txt pgdump-XXX.sql.gz.age | gunzip > restored.sql
ls -lh restored.sql # should be sizable

# 3. Spin up a fresh local Postgres
docker run --name pgdrill --rm -d \
  -e POSTGRES_PASSWORD=test \
  -p 15432:5432 \
  postgres:16

# Wait for ready
sleep 5

# 4. Restore
docker exec -i pgdrill psql -U postgres < restored.sql

# 5. Spot-check
docker exec pgdrill psql -U postgres -c "\l" # list databases
docker exec pgdrill psql -U postgres -d botdb -c "SELECT count(*) FROM users;"
# any other sanity queries on data you'd notice if it was wrong

# 6. Teardown
docker stop pgdrill
rm restored.sql pgdump-XXX.sql.gz.age

```

If anything fails — empty restored file, restore errors, missing tables, zero rows where there should be many — **fix it now**, while you're not under pressure.

Automate the drill

A script to run the drill end-to-end, exiting non-zero if anything is off:

```
/home/julian/scripts/restore-drill.sh:
```

```
#!/usr/bin/env bash
set -euo pipefail

TMPDIR=$(mktemp -d)
trap "rm -rf $TMPDIR; docker rm -f pgdrill 2>/dev/null || true" EXIT

cd "$TMPDIR"

echo "==> Pulling latest dump..."
rclone copy b2:pgdumps-yourdomain/postgres/ . --max-age 2d
DUMP=$(ls pgdump-*.sql.gz.age | sort | tail -1)
[ -n "$DUMP" ] || { echo "No dumps found"; exit 1; }

echo "==> Decrypting..."
age -d -i ~/.config/age/keys.txt "$DUMP" | gunzip > restored.sql
[ -s restored.sql ] || { echo "Decrypt failed or empty"; exit 1; }

echo "==> Starting throwaway Postgres..."
docker run --name pgdrill --rm -d \
  -e POSTGRES_PASSWORD=test \
  -p 15432:5432 \
  postgres:16
sleep 5

echo "==> Restoring..."
docker exec -i pgdrill psql -U postgres < restored.sql

echo "==> Sanity checks..."
docker exec pgdrill psql -U postgres -tc "SELECT datname FROM pg_database WHERE
datistemplate=false;" \
  | grep -q botdb || { echo "botdb missing"; exit 1; }
# Add app-specific checks here

echo " Drill passed."
```

Run it monthly via your calendar reminder. Or wire it up to GitHub Actions on a schedule (but then you're putting the age private key in GitHub secrets, which has its own tradeoffs).

10. Retention policy

How long to keep backups, at each tier.

Tier	Retention	Rationale
Local on VPS	14 days	Covers "I made a bad change last week and didn't notice." Local restores are fast.
Offsite (B2)	90 days	Covers "I made a bad change two months ago that's only now causing problems." Cheap enough to keep this long.
Manual snapshots before risky operations	Indefinite	Before migrations, schema changes, big deploys — take an extra one, tag it, keep until you're confident the change stuck.

TIP. BEFORE ANY DESTRUCTIVE MIGRATION, RUN THE BACKUP SCRIPT MANUALLY FIRST. It produces a fresh, timestamped backup right before the change. If anything goes wrong, you have a known-good point-in-time to restore from.

Pre-deploy backups in CI

Wire this into the deploy workflow if you want zero-thought safety:

```
# .github/workflows/deploy.yml - extend the SSH script
script: |
  cd ~/apm-bot
  # Manual backup before deploy
  /home/deploy/scripts/backup-db.sh
  docker compose pull
  docker compose run --rm migrate
  docker compose up -d bot
  docker image prune -f
```

Every deploy produces a fresh backup. Cheap; no downside.

11. Troubleshooting

"rclone copy" fails with auth errors

Check the config:

```
rclone listremotes          # should list "b2:"
rclone config show b2      # should show your B2 account info
```

Re-run `rclone config` if needed; pick "Edit existing remote."

Cron job runs but produces no output

Likely the cron user doesn't have a TTY-friendly environment. Check:

```
sudo tail -f /var/log/pg-backup.log
```

If the log file is empty or missing, cron isn't even running the job. Check:

```
sudo grep CRON /var/log/syslog | tail -20
```

Common causes: - Cron file in `/etc/cron.d/` has CRLF line endings (from a Windows edit). `dos2unix /etc/cron.d/pg-backup` fixes. - File missing trailing newline. Same fix. - Wrong permissions on the cron file. Should be 644, owned by root. - The script isn't executable (`chmod +x`).

"permission denied" from cron but works manually

The script runs as `deploy` per the cron config. Verify the deploy user can:

```
sudo -iu deploy /home/deploy/scripts/backup-db.sh
```

If this works as `deploy` but cron fails, check `$PATH` (cron has a minimal one). Use absolute paths in the script (`/usr/bin/docker`, `/usr/bin/gzip`) if there's any ambiguity.

Restored database has different data than expected

Most often: you restored an older dump than intended. Check:

```
ls -la ~/backups/  
# Find the most recent one before whatever-broke
```

`pg_dumpall` is a point-in-time snapshot — if data changed between the snapshot and "now," you'll only have what was there at snapshot time.

age decryption fails

```
age: failed to decrypt: no identity matched any of the recipients
```

Causes: - Wrong private key. Check that the `~/.config/age/keys.txt` file matches the public key the backups were encrypted to. - Public key changed at some point and old backups were encrypted to a different key. Keep all your age keys; they're cheap to maintain.

To inspect what public keys a backup was encrypted to:

```
age -d -i nonexistent-key.txt backup.age 2>&1 | grep "recipients"
```

(It'll fail decryption but print what recipients the file accepts.)

Backups are slow / hitting Backblaze daily-cap

Free-tier B2 has a daily 1GB download cap (uploads are unlimited). If you exceed it during restores, the cap is hit. Upgrade to paid (still pennies/month) or schedule restores around the cap reset.

12. Alternatives considered

Storage backends

- **Backblaze B2** (this guide) — cheapest credible offsite. Default.
- **AWS S3** — more expensive (\$0.023/GB-month standard); reconsider if you're already in AWS or need cross-region replication.
- **Cloudflare R2** — S3-compatible, zero egress fees. Reconsider if you do lots of restores (egress matters).
- **Hetzner Storage Box** — flat-rate FTP/SMB storage, cheap (€3/month for 1TB). Reconsider for personal backups if you're already using Hetzner.
- **Your own NAS at home** — works if you have one. Off-site via your home internet's upload bandwidth.

- **Self-hosted MinIO on another VPS** — adds complexity for marginal benefit at personal scale.

Encryption

- **age** (this guide) — modern, simple, no key servers. Default.
- **GPG / gpg-agent** — older standard, more options, larger surface area. Reconsider if you have an existing GPG setup.
- **rclone's built-in encryption (`crypt` remote)** — encrypts at the rclone layer. Easier than age. Tradeoff: tied to rclone's specific scheme; less portable.
- **Server-side encryption only** (B2's at-rest encryption with their keys) — adequate against random attackers; useless against a B2 employee or a compromised B2 account. Skip.

Backup methodology

- `pg_dumpall` **cron** (this guide) — simplest reliable approach. Daily snapshots. RPO ~24h.
- `pg_dump --create -d <db>` **per database** — same idea, finer-grained. Reconsider when restoring one db from a multi-db dump becomes annoying.
- **WAL streaming / point-in-time recovery (PITR)** — restore to any second within the retention window. Reconsider when 24-hour RPO is too coarse. Significantly more complex.
- `pg_basebackup` + **WAL archive** — physical-level backup; restores faster than `pg_dump` for large databases. Reconsider when DB is >50GB or restores are too slow.
- **Logical replication to a standby** — near-zero RPO. Reconsider when you have an actual HA requirement; overkill for personal use.
- **Filesystem snapshots (ZFS / btrfs)** — beautiful when set up; requires a snapshot-aware filesystem on the VPS. Reconsider if your VPS supports it.

Application file backups

For things like user-uploaded images, sqlite databases inside containers, etc.:

- **Add a `tar` of the volume to the backup script.** Simple.
- **Restic** — deduplicating backup tool. Better for many small files. Reconsider when total file size is large and similar files are common.
- **Bup, BorgBackup** — similar to Restic. Older.

13. Quick reference

One-time setup checklist

- [] Backblaze B2 account, bucket created.
- [] Application key with read+write to the bucket.
- [] `rclone` installed and configured on the VPS.
- [] `age` installed; keypair generated.
- [] **Private age key stored in 1Password / offline copy.**
- [] Public age key noted (you'll embed it in scripts).
- [] `~/.config/age/keys.txt` permissions: `0600`.
- [] `~/.config/rclone/rclone.conf` permissions: `0600`.

- [] Backup scripts in `/home/deploy/scripts/`.
- [] Cron entries in `/etc/cron.d/pg-backup`.
- [] First manual run successful.
- [] First B2 upload verified (`rcclone lsf b2:...`).
- [] Monthly drill date on the calendar.

Daily / weekly automation

What	Schedule	Script
Postgres dump + offsite	Nightly 03:17 UTC	<code>backup-db.sh</code>
Configs tarball + offsite	Weekly Sundays 03:27 UTC	<code>backup-configs.sh</code>

Restore drill (monthly)

```
~/scripts/restore-drill.sh
```

Emergency: restore a database to the live VPS

```
ls ~/backups/ # find the snapshot
age -d -i ~/.config/age/keys.txt ~/backups/pgdump-XXX.sql.gz.age | gunzip > /tmp/restore.sql
# stop dependent apps
docker exec -i infra-postgres-1 psql -U postgres < /tmp/restore.sql
shred -u /tmp/restore.sql
# start dependent apps
```

Disaster recovery: rebuild from zero

See §8. Roughly:

1. Provision a fresh VPS (per `vps-from-zero`).
2. Restore age key.
3. Restore configs tarball.
4. Bring up infra Postgres.
5. Restore latest DB dump.
6. Bring up Caddy + apps.
7. Update DNS.